

Reinforcement Learning.

실습 1 — 정비소 배치 문제

Monday, August 10, 2026

DS² 과정

Intelligent Motion Lab (Seoul National University)



대기 중인 차를 어느 정비소로 보낼지 고릅니다



어떻게 짝지어도 되지만, 걸리는 시간이 달라집니다



정비소 3개, 대기 3칸, 차 100대

매 틱마다 하는 일

- 대기 차량 하나를 골라 빈 정비소에 배치
- 또는 아무것도 하지 않고 대기

정해진 것

- 대기 공간은 **최대 3대**, 4대째는 못 받음
- 차량마다 **인내도**가 있고 0이 되면 떠남

100대를 최소한의 시간에 정비 완료하는 것이
목표

차량마다 크기·연식·손상도가 다릅니다

차량 정보

- 차체 크기 3.0 ~ 5.0
- 연식 10 ~ 25
- 손상도 0.0 ~ 1.0
- 인내도 (틱)

정비소 특성 (문제 2)

- A 22~23 틱
- B 21~24 틱
- C 10~27 틱, 크기 ≤ 4.0 이면 50% 감소

점수는 소요 시간에 이탈 벌점을 더한 값입니다

$$\text{점수} = \text{소요 틱} + 50 \times \text{이탈 대수}$$

- 낮을수록 좋습니다
- 빨리 처리하되, 기다리다 떠나게 하면 안 됩니다

문제는 두 개입니다

	문제 1	문제 2
도착 간격	0~30 틱	없음 (매 틱)
인내도	30	100
A	20~25	22~23
B	20~25	21~24
C	20~25	10~27, 크기 ≤ 4.0 시 50% ↓
베이스라인	915	630

코드 익히기

먼저 돌려보고, 읽고, 어디를 고칠지 확인합니다

파일 구조와 베이스라인

```
env/_core.py          시뮬레이터 (수정 금지)
env/garage_1.py        문제 1
env/garage_2.py        문제 2

student/solution.py ← 여러분이 쓰는 곳

config/ppo_config.json
train.py test.py verify_env.py
```

- `env/` 는 수정 불가입니다
- 여러분이 고치는 것은 `solution.py` 하나

```
$ python env/garage_1.py      # 문제 1  ->  913.2
$ python env/garage_2.py      # 문제 2  ->  631.4

[GarageEnv_2] baseline over 100 episodes
  score   : 631.4  (sd 21.6)
  best / worst : 587 / 676
```

- 빈 정비소가 있으면 맨 앞 차를 넣는 규칙의 점수입니다
- 이 숫자가 오늘의 기준선입니다
- 시드에 따라 값이 조금 달라질 수 있습니다

이 규칙 8줄이 이미 거의 최적입니다

```
def baseline_action(env):  
    # 빈 정비소가 있으면  
    # 대기열 맨 앞 차를 A, B, C 순으로 배치  
    if env.waiting_area:  
        for i, st in enumerate(['A', 'B', 'C']):  
            if env.repair_status[st] is None:  
                return i + 1  
  
    return 0
```

베이스라인 (문제 2)

정비소 가동률	92.9%
100대 정비에 든 총 작업량	1859 틱
정비소 3개로 나누면	620 틱
실제 소요	631 틱

더 부지런히 해서 얻을 건 11틱뿐입니다

환경을 직접 만들어 들여다봅니다

```
from env.garage_2 import GarageEnv_2

env = GarageEnv_2(seed=0, need_obs=False)
env.reset(seed=0)

for _ in range(40):          # 40틱 흘려보내기
    env.step(0)              # 0 = 아무것도 안 함

for i, car in enumerate(env.waiting_area):
    print(i, car.size, car.year, car.damage,
          env.car_patience[car.id])
```

```
0  3.08  25  0.40  60
1  4.57  19  0.97  60
2  4.01  19  0.14  61
```

- `need_obs=False` 면 `solution.py` 없이도 만들어집니다
- 0번 차는 크기 **3.08** — C 에 넣으면 유리합니다
- 1번 차는 4.57 — C 에 넣어도 이득 없습니다

쓸 수 있는 정보와 행동 10가지

```
env.waiting_area          # 대기 Car 리스트 (0~3대)
    car.size  3.0~5.0    car.year  10~25
    car.damage 0.0~1.0

env.car_patience[car.id]  # 남은 인내도
env.repair_status['A']     # None 이면 비어 있음
                           # (car, 남은틱, 할당틱)

env.max_waiting  3        # 대기 공간
env.max_patience          # 이 문제의 인내도 최댓값
```

- 100 을 쓰지 말고 `env.max_patience` 로 나누세요

정의

```
0          대기 (아무것도 안 함)
1, 2, 3    0번 차를 A, B, C 에
4, 5, 6    1번 차를 A, B, C 에
7, 8, 9    2번 차를 A, B, C 에
```

알아둘 것

- 이미 차가 있는 정비소를 고르면 **아무 일도 안 일어남**
- 비어 있는 슬롯을 골라도 마찬가지
- 즉 **무효 행동을 피하는 것도 학습 대상**

여러분이 채우는 두 함수와 event

```
OBS_DIM = 0      # TODO 1: 관측 벡터 길이

def get_observation(env):
    vec = np.zeros(OBS_DIM, dtype=np.float32)
    # TODO 2: 에이전트가 볼 것을 채운다
    return vec

def compute_reward(env, event):
    # TODO 3: 무엇을 잘한 것으로 볼지 정한다
    return 0.0
```

event 가 이번 스텝에 무슨 일이 있었는지 알려줍니다

```
event['assigned']    # 정비소에 넣었다
event['invalid']     # 점유된 정비소를 골랐다
event['expired']     # 이탈한 차량 수
event['finished']    # 정비가 끝난 차량 수
event['done']        # 에피소드가 끝났다
```

- 관측은 모두 **[0, 1]** 범위 — 벗어나면 즉시 오류로 알려줍니다
- 안내의 기본값은 무효 배치 **-1**, 이탈 **-100** — 이것만으로는 부족합니다

학습·평가 명령과 진행 출력

```
python train.py --level 2
python test.py --level 2 --baseline
python verify_env.py

# 학습 곡선
tensorboard --logdir=tensorboard
```

- `test.py` 는 고정 시드 100 에피소드 평균
- 같은 모델이면 항상 같은 결과
- `verify_env.py` 는 환경이 원본인지 확인

베이스라인 631.2 (낮을수록 좋음)

진행	스텝	경과	잔여	평균보상	점수	대비
10%	50,000	0:12	1:56	-5.1	678.6	-7.5%
50%	250,000	1:06	1:06	97.8	572.4	+9.3%
100%	500,000	2:12	0:00	97.5	553.0	+12.4%

학습 시간 2분 13초 (3745 steps/s)

- **평균보상** 은 여러분이 설계한 것 · **점수추정** 은 실제 지표
- 두 열이 같이 움직이지 않으면 보상 설계를 다시 봐야 합니다

관측과 보상 배선

먼저 학습이 도는지만 확인합니다

관측과 보상을 가장 작게 채워봅시다

```
OBS_DIM = 3 + 3      # 슬롯 점유 3 + 정비소 가동 3
```

```
def get_observation(env):  
    vec = np.zeros(OBS_DIM, dtype=np.float32)  
    n = len(env.waiting_area)  
    for i in range(env.max_waiting):  
        vec[i] = 1.0 if i < n else 0.0  
    for j, st in enumerate(['A', 'B', 'C']):  
        busy = env.repair_status[st] is not None  
        vec[3 + j] = 1.0 if busy else 0.0  
    return vec
```

```
def compute_reward(env, event):  
    reward = 0.0  
    if event['assigned']: reward += 1.0    # 놀리지 마라  
    if event['invalid']:  reward -= 0.5    # 헛발질 마라  
    return reward
```

- 차량 속성은 **아직 넣지 않습니다** — 상식적으로 보이지만 빠진 것이 있습니다

돌려보면 실패합니다 — 학습을 안 했기 때문입니다

```
$ python train.py --level 1
...
$ python test.py --level 1 --baseline
```

baseline	915.4		
PP0 agent	3753.5	개선	-495.4%

- 베이스라인의 **4배**, 이탈 **37.9대**

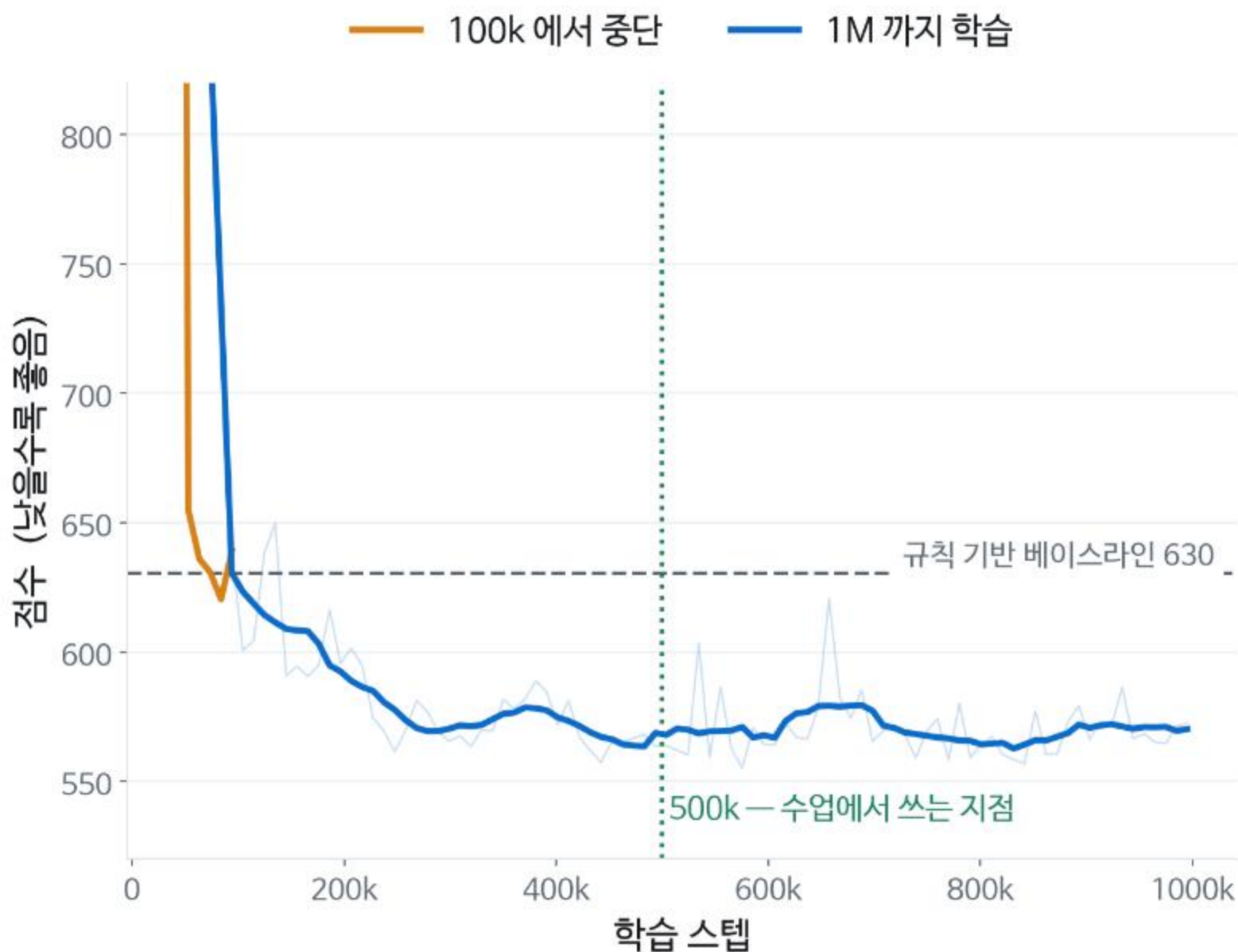
```
// config/ppo_config.json
"total_timesteps": 100,      ← 이것
"n_steps": 2048,
"ent_coef": 0.0,
```

- rollout 한 번 돌고 끝 → **정책이 사실상 무작위**

하이퍼파라미터 하나가 학습을 원천 차단할 수 있습니다

학습량을 늘리면 베이스라인에 도달합니다

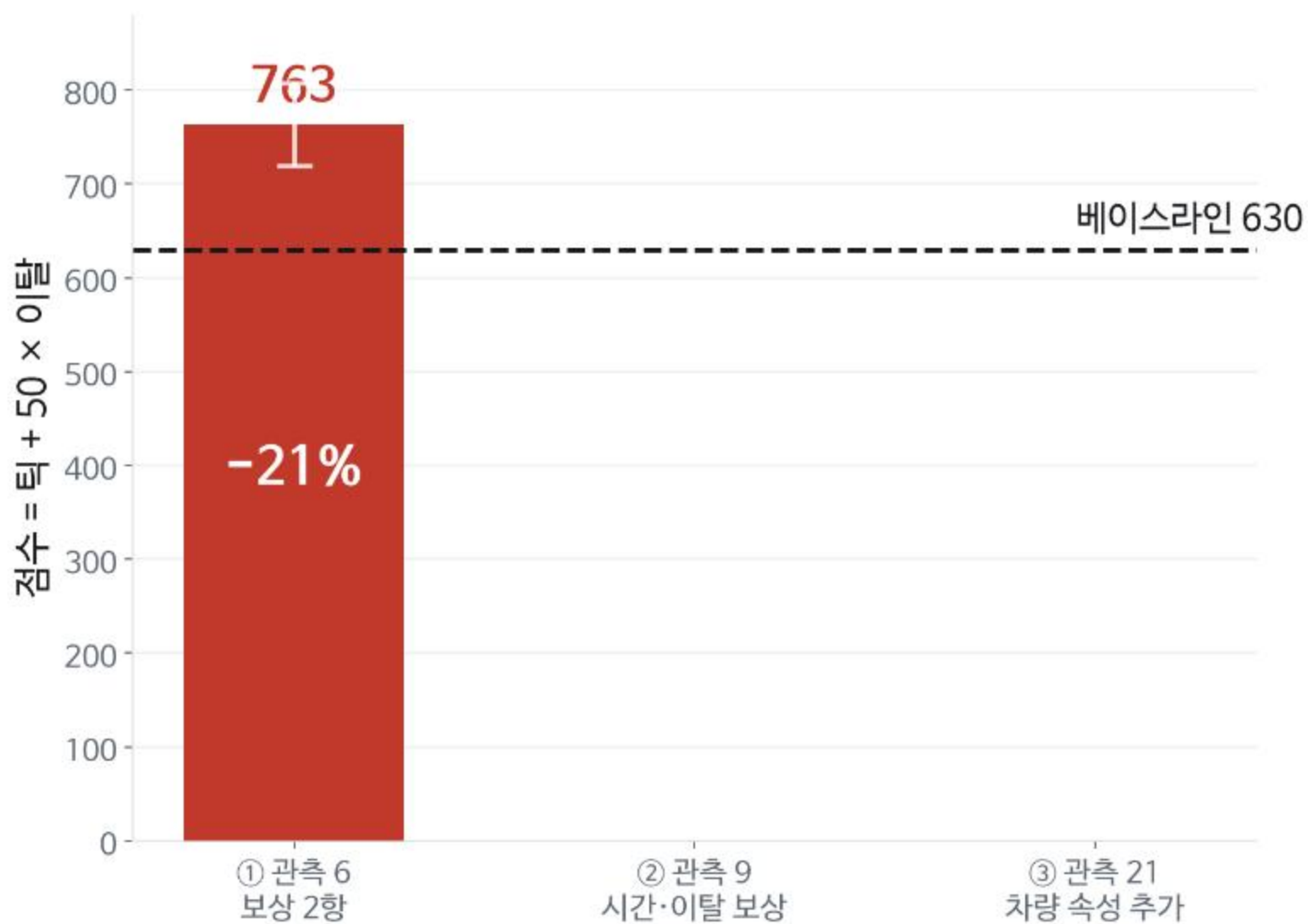
- 100k 면 베이스라인 수준
- 500k 면 넘어섭니다
- 500k 이후는 평평합니다



보상 설계

목표에 있는 것이 보상에도 있는가

같은 코드로 문제 2를 돌리면 20% 나뉩니다



보상은 오르는데 점수는 베이스라인 아래로 못 내려갑니다



보상에 시간이 없었습니다 — 넣어봅시다

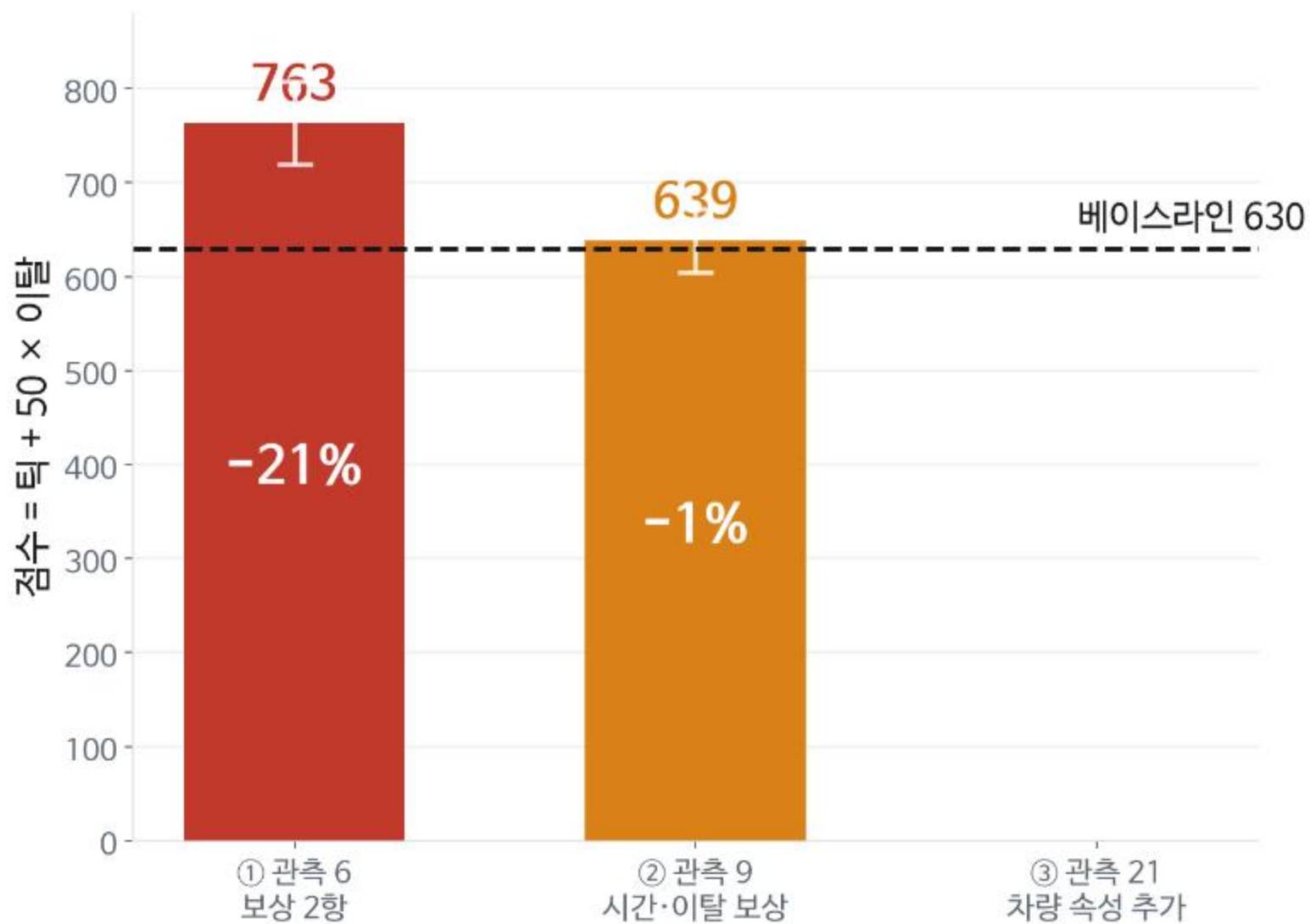
목표는 총 소요 시간인데
보상에는 시간 항이 없었습니다

- 배치하면 +1 — 빨리 하든 늦게 하든 같은 +1
- 이탈에도 벌점이 없었습니다

```
def compute_reward(env, event):  
    reward = -0.02                # 매 틱 시간 비용  
  
    if event['assigned']:  
        reward += 1.0  
    if event['invalid']:  
        reward -= 0.5  
  
    reward -= 20.0 * event['expired'] # 이탈  
  
    if event['done']:  
        reward += 10.0            # 완주 보너스  
  
    return reward
```

- 안내문의 -100 대신 **-20**
- 다른 항이 ± 1 규모인데 -100 은 너무 큼니다

이탈은 잡혔지만 베이스라인은 못 넘습니다



관측 설계

관측에 없는 정보는 쓸 수 없습니다

에이전트는 차체 크기를 볼 수 없었습니다

C 정비소: 10~27 틱,
단 차체 크기 ≤ 4.0 인 차량은 50% 감소

지금 에이전트가 차체 크기를 볼 수 있습니까?

에이전트가 본 것

[1, 1, 1,	슬롯 점유
1, 0, 1]	정비소 가동

실제로 필요했던 것

0번 차:	크기 3.2	→ C 에 넣으면 유리
1번 차:	크기 4.8	→ C 에 넣어도 이득 없음
2번 차:	크기 3.5	→ C 에 넣으면 유리

차량 속성을 관측에 넣습니다

```
OBS_DIM = 3 + 3 + 3 + 3 * 4    # 21

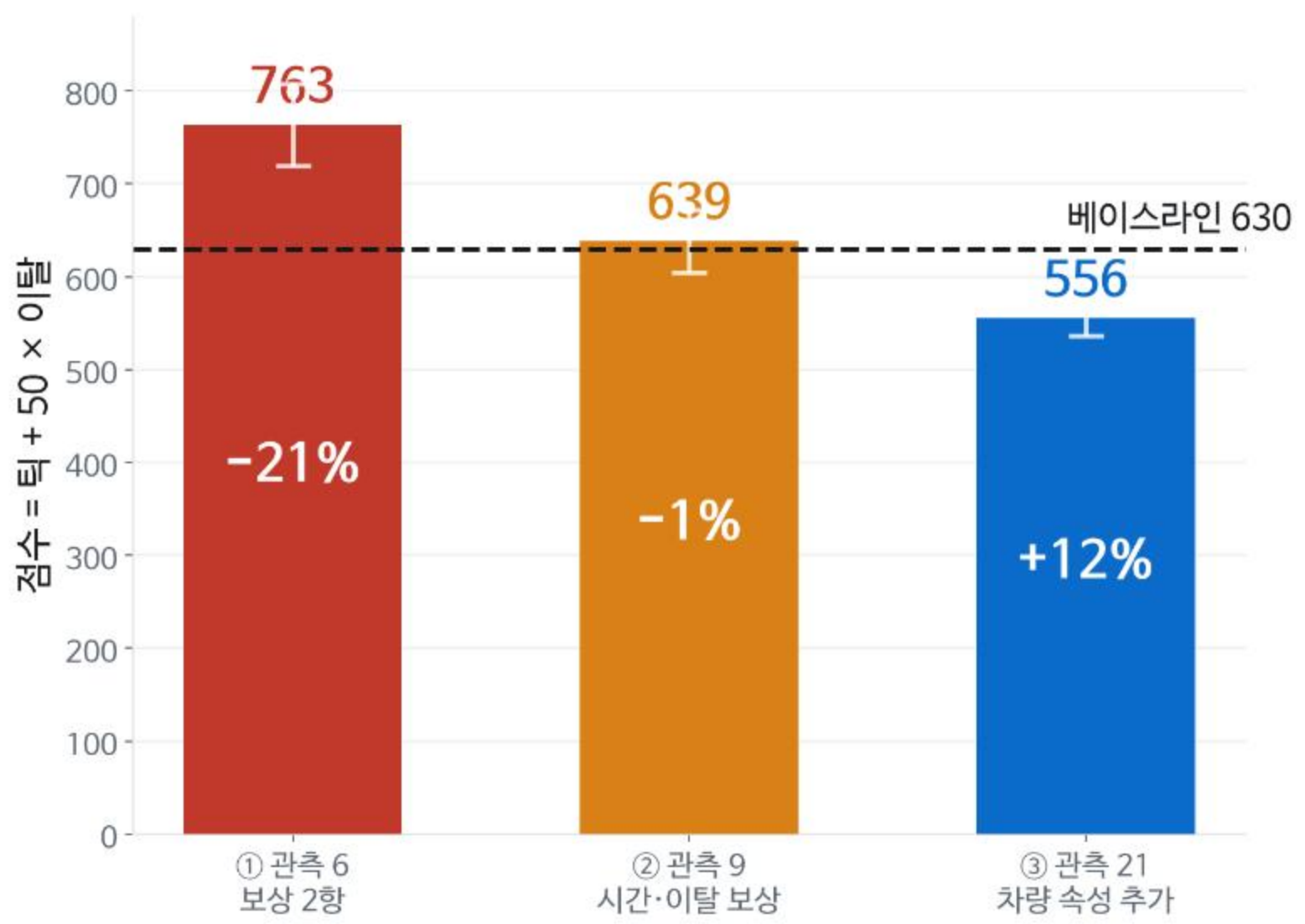
# ... 앞의 9개는 그대로 ...

for i in range(env.max_waiting):
    if i >= len(env.waiting_area):
        continue
    c = env.waiting_area[i]
    pat = env.car_patience[c.id]
    b = 9 + i * 4

    vec[b+0] = (c.size - 3.0) / 2.0
    vec[b+1] = (c.year - 10.0) / 15.0
    vec[b+2] = c.damage
    vec[b+3] = pat / env.max_patience
```

- 대기 차량 3대 × 속성 4개
- 모두 [0, 1] 로 정규화

관측만 바뀌 베이스라인을 넘었습니다



판단 근거가 있는 정책은 흔들리지 않습니다



- 학습 중 실제 지표를 5 에피소드씩 실측한 곡선입니다

보상 설계보다 관측 설계가 먼저입니다

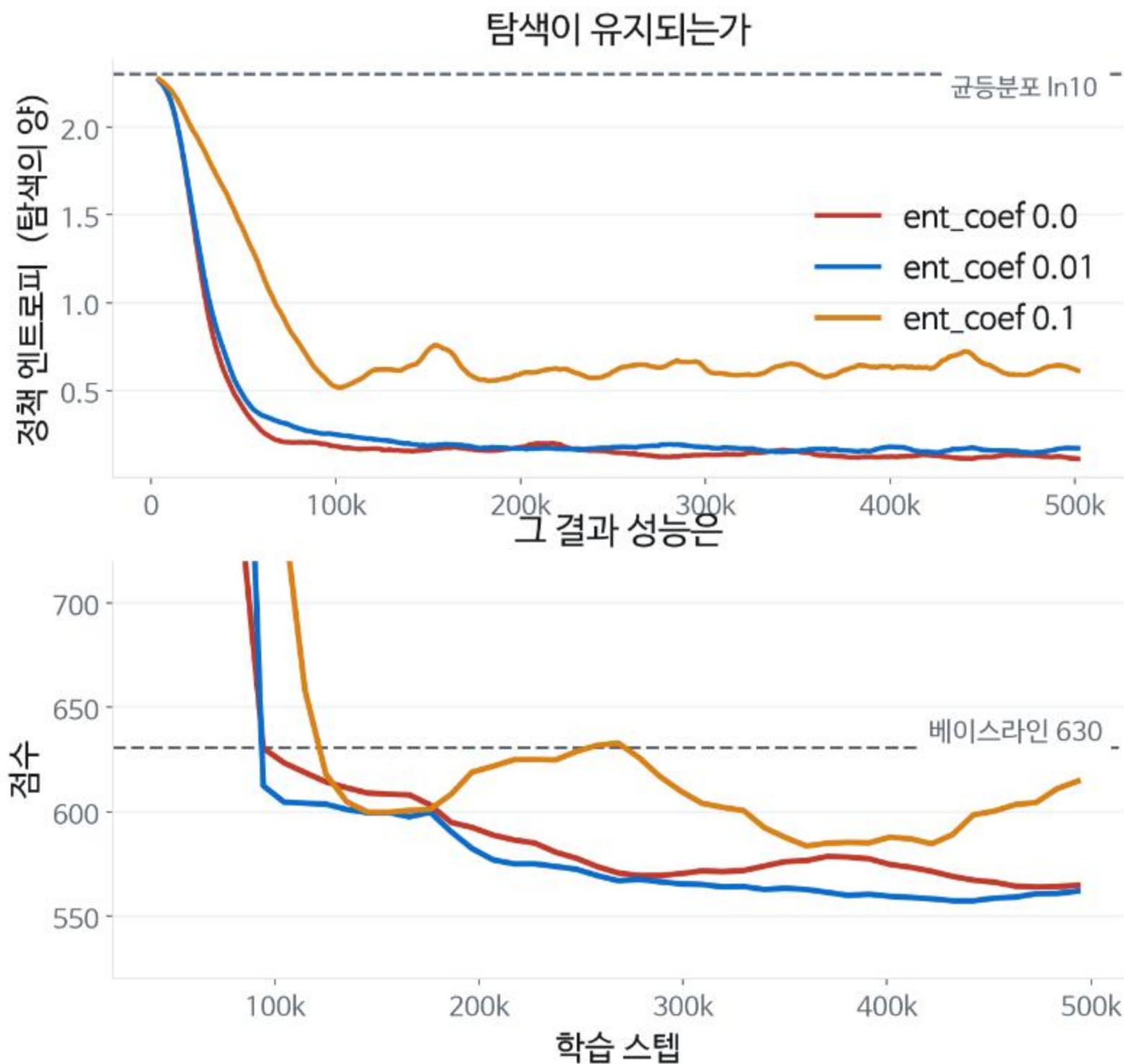
관측에 없는 정보는
어떤 보상으로도, 어떤 학습량으로도
보완되지 않습니다

관측	점수	대비
6차원 · 보상 2항	763	-21%
9차원 · 시간·이탈 보상	639	-1%
21차원 · 차량 속성	556	+12%

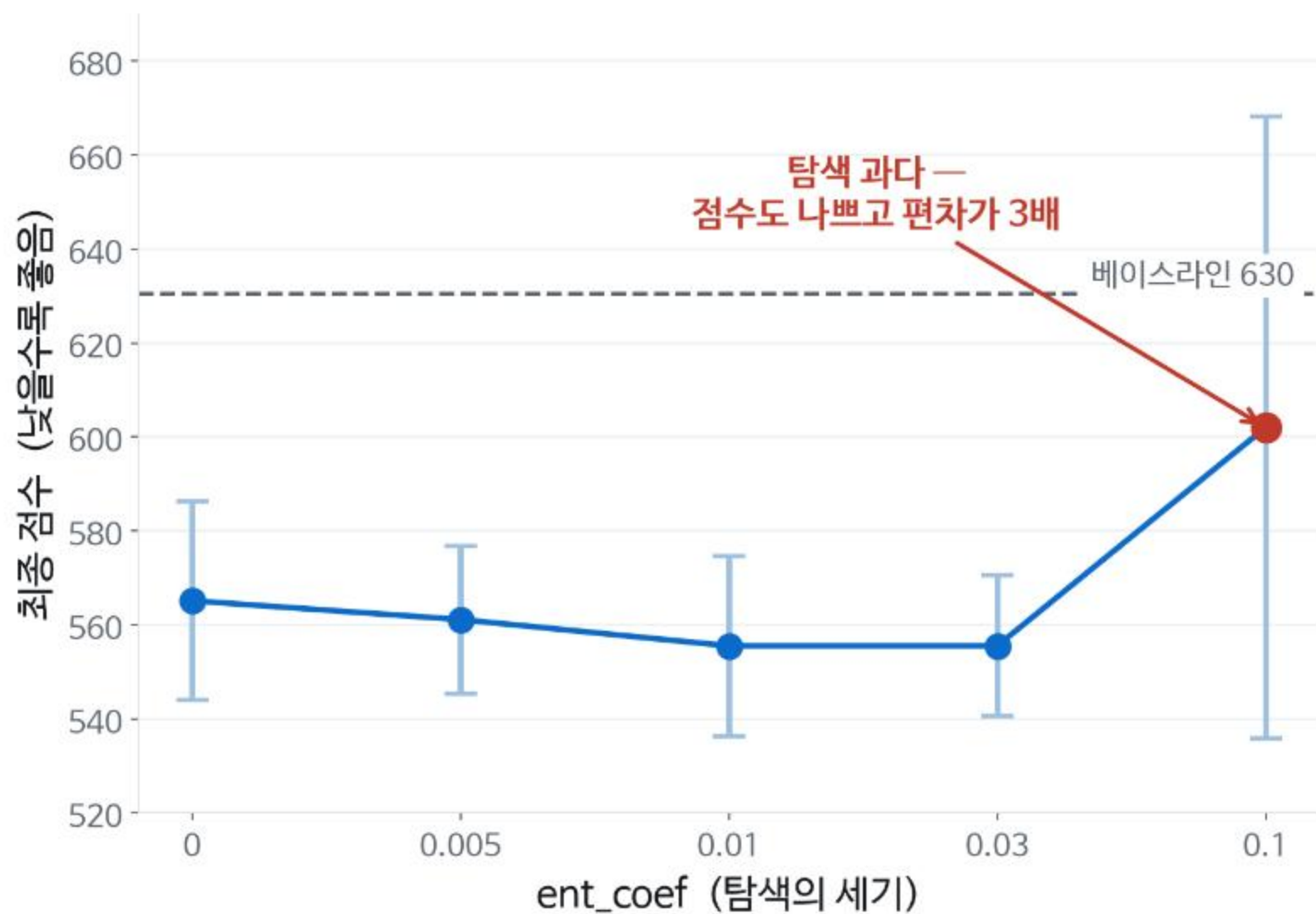
하이퍼파라미터 튜닝

무엇을 만질지 아는 것

탐색이 과하면 확실히 나빠집니다



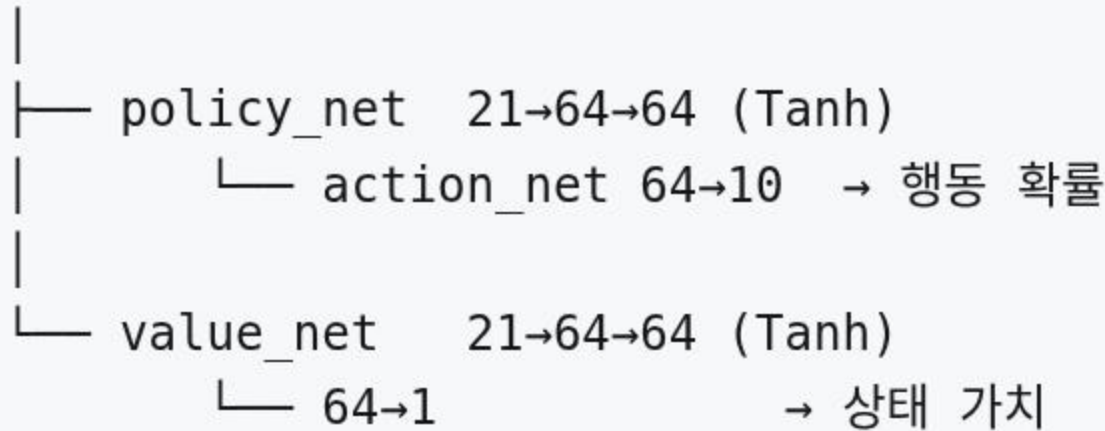
적당한 범위는 넓고, 과다는 위험합니다



- 오차 막대는 100 에피소드의 표준편차

정책 내부와, 무엇을 만질지

관측 21차원



파라미터 11,851개

- `OBS_DIM` 을 바꾸면 **입력층이 바뀝니다**
- 학습 중엔 확률로 샘플링, 평가는 `argmax`

이 문제에서 중요했던 것

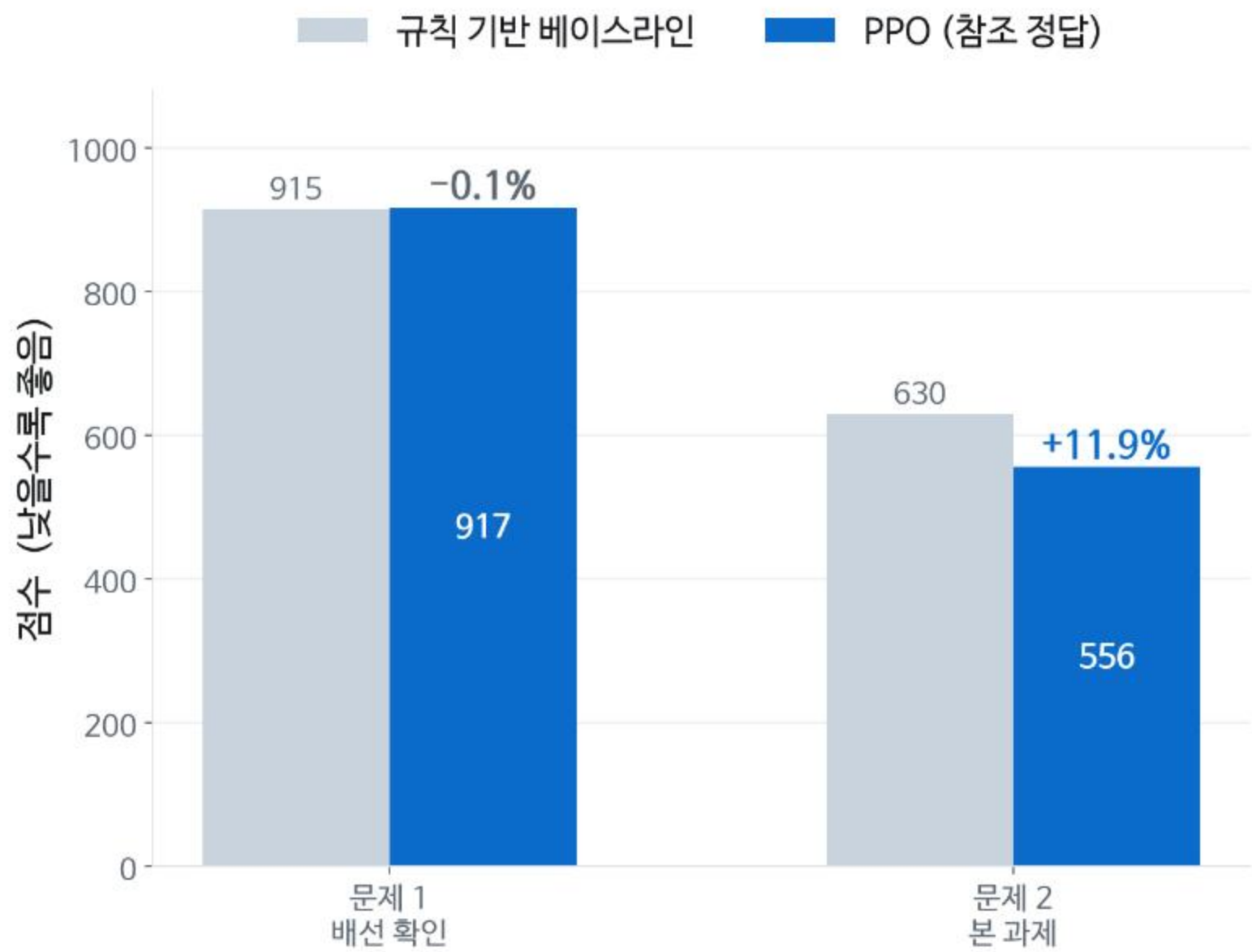
- `total_timesteps` — 부족하면 전부 무의미
- `ent_coef` — 과하면 확실히 나쁨

중요하지 않았던 것

- `n_steps` (100 / 512 / 2048 차이 없음)
- 신경망 크기

한 번에 하나만 바꿔서 비교합니다

두 문제를 한 눈에



오늘 남는 네 가지

1. 관측이 비면 아무것도 배울 수 없다
2. 하이퍼파라미터 하나가 학습을 원천 차단할 수 있다
3. 목표에 있는 것이 보상에 없으면 학습되지 않는다
4. 관측에 없는 정보는 활용될 수 없다